Lifelong Learning Techniques

for an Origami Robot

by

E. De Vroey

Student number: 4685156

TU Delft

Contents

1 Introduction **1**
- 1.1 Application to soft (origami) robots . . . 1
- 1.2 Challenges . . . 1

2 Background and definition **3**

3 Blackbox methods **5**
- 3.1 Learning control policies . . . 5
- 3.2 Learning the dynamic model . . . 6
- 3.3 Incremental learning. . . . 7
- 3.4 Summary . . . 8

4 Learning architectures **9**
- 4.1 Traditional adaptive control architectures . . . 9
- 4.2 Dual architectures . . . 9
- 4.3 Architectural expansions . . . 10

5 Storage and retrieval **13**
- 5.1 Knowledge bases . . . 13
- 5.2 Retrieval and Detecting distributional shifts . . . 13
- 5.3 Maintaining the knowledge base . . . 13

6 Discussion and conclusion **15**

1 Introduction

1.1. Application to soft (origami) robots

1.2. Challenges

(Kim et al., 2021; Lesort et al., 2020).

> How can lifelong learning methods be employed to manage the evolving dynamics due to wear and tear in the control of a soft origami robot over its operational lifespan?

2

Background and definition

(Annaswamy & Fradkov, 2021; Landau et al., 2011; Lesort et al., 2020).

3

Blackbox methods

The term ‘blackbox’ in the context of lifelong learning techniques describes models that are inherently ‘data-driven’, requiring vast datasets to predict outcomes while often lacking in interpretability. These methods differ from traditional approaches, where clever architectural approaches or intuitive mathematical tricks might be exploited. Instead, so-called blackbox methods trade transparency for performance, as they process and learn from information in hard-to-interpret yet effective ways.

In this chapter, an overview is provided of how blackbox methods can be used as either building blocks for lifelong learning or to solve the lifelong learning challenge as complete solutions.

3.1. Learning control policies

A perfect example of a blackbox method is given by Alessi et al. (2023). In their study, they used Proximal Policy Optimization (PPO) (Schulman et al., 2017) to obtain a feedforward neural network (FNN) that can generalize over new dynamics and tasks. The controller is a blackbox: a desired trajectory, several measurements of the robot’s state, and other values of interest, such as the error of robot’s end-effector tip, are used as inputs, resulting in a control action. It essentially acts in a manner akin to a feedback controller using an inverse dynamics model, only in this case it is unknown what is happening *under the hood*.

The previous example is *“perfect”* in the sense that it perfectly embodies the simplistic nature of blackbox methods, however, it is worth noting that other techniques, such as more complicated architectures or combinations of multiple models, can be used in combination with these blackbox methods. Such architectural modifications will be discussed in chapter 4.

Apart from Alessi et al. (2023), several other examples of blackbox policies can be found in the literature.

In Zhou et al. (2022), two policies are learned for the control of a small humanoid robot. Exact details on the inputs and outputs of the controllers are not provided, but both policies are probabilistic in nature.

Thuruthel et al. (2019) use guided policy search (Levine & Koltun, 2013) over trajectory samples containing control actions for given regions in the state-space of a soft robotic manipulator. The resulting control policy is represented as a FNN which takes in current and previous states, as well as a desired target state, and returns the appropriate control action.

For the control of a soft robotic arm, in Nazeer et al. (2023), the policy is represented by a multi-layer perceptron (MLP) which takes in the arm’s current state[1] and outputs a control

[1]Rather, the sum of the state and a correction term, resulting from a separate network (section 4.3), but that is not

action. The policy is trained using the Soft Actor-Critic (SAC) (Haarnoja et al., 2018) reïnforcement learning algorithm.

In Lu et al. (2020), a more complicated technique is used. A probabilistic 'skill' distribution $p_\psi(z \mid s)$ and a neural network-based policy $\pi_\theta(a \mid s, z)$ are independently and simultaneously learned using SAC. Both models are trained on rollouts of state transitions generated by a dynamics model. Once learned, a model-based planning algorithm called Model Path Predictive Integral (MPPI) (Williams et al., 2015) is used to find the optimal skill sequence in the skill distribution p_ψ, resulting in executable actions through the policy π_θ.

3.2. Learning the dynamic model

One way of having continually adapting control for soft robots is by implementing a (traditional) control scheme that uses a dynamic model (either forward or inverse) of the robot and then to incrementally update that model. The idea of learning a dynamic model for soft robots has advantages even outside the context of lifelong adaptation: many soft robots have such a large number of degrees of freedom and complex and nonlinear dynamics, that analytically deriving a dynamic model is both cumbersome and often insufficiently accurate (Gillespie et al., 2018; Giorelli et al., 2015; Kim et al., 2021; Thuruthel et al., 2019).

Blackbox methods also allow for learning both forward and inverse dynamic models and can thus be used in several control schemes. When learning a forward model, the robot's actions and resulting states can be used as the inputs and outputs, respectively, this is so-called *direct modeling* (Nguyen-Tuong & Peters, 2011b). In the case of learning inverse dynamic models, such direct approaches might not always be applicable, due to the multiple solutions that exist for the inverse dynamics of robots with redundant degrees of freedom. Instead, an *indirect-modeling* approach can be taken, where the model is trained to minimize the error of the feedback controller (Nguyen-Tuong & Peters, 2011b).

The form that the dynamic model takes can vary. For example, in Gillespie et al. (2018), a neural network is trained to predict the next state of the robot, given the current state and a control action, the analytical gradients of these outputs w.r.t. the inputs are made available during the process of backpropagation and can be used in matrices A and B to form a more interpretable state-space system. In Thuruthel et al. (2019) on the other hand, a forward model is learned for open-loop control, using the control action, the current state, and the previous state as input, and the next state as output to train a recurrent neural network. This forward model is kept in its more abstract blackbox form as a recurrent neural network (RNN), and is used in combination with trajectory optimization to generate trajectories in open-loop control.

Since the lifelong adapting nature of the techniques is embedded in the dynamic model, the choice of control scheme in which the learned dynamic model is used can vary from implementation to implementation. To illustrate, the aforementioned matrices A and B from Gillespie et al. (2018) are used in a model predictive control setting, while the generated trajectories from Thuruthel et al. (2019) are used as samples to train a blackbox control policy (the dynamic model is in essence an intermediary step to create the blackbox policy mentioned in section 3.1). More examples several blackbox dynamic models and their control scheme can be found in the literature.

Nazeer et al. (2023) represent a forward dynamic model of the soft robotic arm as an RNN model that uses the two preceding states and three preceding control actions to predict the next state. The model is then used with the previously discussed MLP policy (section 3.1).

Lu et al. (2020) was discussed in section 3.1 where it was briefly mentioned that training rollouts for the policies were generated using a dynamic model. This model is in fact also a data-driven model, being an ensemble of neural networks and predicting the next state s'

relevant for the basic idea of the policy.

based on the current state s and a control action yielded by the policy π_θ.

In and Nguyen-Tuong and Peters (2011a), the inverse dynamic model is represented by a Gaussian process (a type of probabilistic model) and is learned through Local Gaussian Process Regression (Nguyen-Tuong et al., 2008, 2009). The models are employed in a feedforward control scheme, supplemented by a feedback term that adjusts the control action outputted by the dynamic model.

redundant references?

Add paragraph on Gijsberts and Metta (2011)

Romeres et al. (2016) represent an inverse dynamic model of a robotic arm as a Gaussian process, but also experiment with using the arm's rigid body dynamics in several ways to enhance the model's performance. By incorporating the information from the robot's physical parameters into both the mean function and the covariance kernel of the Gaussian process, they develop (a) semi-parametric model(s) that are used in a feedforward manner where a feedback controller adds small corrections to the control actions outputted by the dynamic model.

Contrary to blackbox policy learning methods, the adaptation to new tasks is not included in the learning process of the dynamic model, but is entirely dependent on the control policy. In case control policies have been defined in advance for several tasks, these task settings might serve as a test to verify that the dynamic model generalizes well over these different settings. In situations where tasks are not known in advance, a control scheme such as an incrementally updated (inverse) dynamics model in some form of a feedback control loop might serve for simple applications (Nguyen-Tuong & Peters, 2011a). For more complex tasks, a more intricate control system might be warranted.

rethink order of paragraphs?

3.3. Incremental learning

In section 3.1 and 3.2 an overview was provided of how blackbox machine learning techniques can be used to model either control policies or dynamic models (to be used in control policies). The choice of machine learning method can be important when it comes to the desired incremental nature of the learning techniques. To illustrate, consider these two main strategies in continually adaptive control:

1. All training is performed offline (after gathering sufficient data), and the robot is now completely capable of immediately adapting to changing dynamics it will encounter during its operational lifespan.

2. Training is performed incrementally as data comes in, which can potentially be done in real-time.

In both scenarios, the control of the robot would be continually adaptive. However, only strategy 2 aligns with the incremental objectives of the adaptive control challenge. In addition, the complex dynamics and many degrees of freedom of soft robots might make it difficult to accurately represent all possible degradations in dynamics (such as damages) or state-space configurations of the robot, potentially causing an imbalance in the training dataset of strategy 1. Incrementally learning to adapt the control policy or dynamic model is thus far more desirable, since it can cope with unforeseen circumstances.

Not all the previously discussed methods are - or have the possibility to be - implemented in an incremental manner. As an example, the two policies from Zhou et al. (2022) mentioned in section 3.1 are trained sequentially: one online and the other offline. Each of these policies required different training/optimization algorithms for their specific setting, in this case Maximum a Posteriori Policy Optimization (MPO) (Abdolmaleki et al., 2018) for the online interaction setting, and Critic Regularized Regression (CRR) (Wang, Novikov, et al., 2020) for the offline distillation setting (see section 4.2).

To learn the dynamic model from Gijsberts and Metta (2011), the authors implement Ridge Regression (a linear regression method that aims at keeping the parameters as small as possible) incrementally, whereas normally it is used in a batch-learning setting. They achieve this by employing a modified numerical approach based on the QR algorithm (Sayed, 2008), a technique in linear algebra that can process new information incrementally using Givens rotations (a process that is particularly efficient for online learning scenarios) (Björck, 1996). This enables real-time learning as new data comes in without recalculating the full model.

Lu et al. (2020) implement both offline and online training, with the potential for the entire method to be implemented online. In fact, it is well known that SAC - the algorithm used for updating the skill-space distribution and the policy - is very suitable for online updates. When it comes to the dynamic model, they mention that is incrementally updated through interactions with the environment, but do not provide any details on the algorithm used for training the dynamic model.

In Nguyen-Tuong and Peters (2011a) the authors not only update their model incrementally, but they also manage to do it in real-time. This is achieved by using incremental Gaussian process regression algorithm, which uses only a select number of data points to fit a model. In addition, they introduce an efficient way to incrementally update the dictionary of datapoints in a way that it maintains the most informative points for a given budget. These two techniques are key to reducing the computational demand and allowing for real-time updates.

The semi-parametric models from Romeres et al. (2016) are learned using the Recursive Least-Squares algorithm (Landau et al., 2011), which inherently updates the model incrementally. Much alike Gijsberts and Metta (2011), this method uses a numerical trick from the field of adaptive filtering (Cholesky-based updates) to improve efficiency and enable updates in real-time.

3.4. Summary

In Table 3.1, a clear comparison of blackbox methods in lifelong learning is provided. This summary aims to give a straightforward reference, highlighting the strengths of blackbox methods and their role in lifelong learning.

Reference	Dynamic model	Control policy	Incremental
Gillespie et al. (2018)	SS-matrices	MPC	–
Gijsberts and Metta (2011)	Linear model in random feature space	–	✓
Thuruthel et al. (2019)[2]	RNN	Open-loop + trajectory optimization	–
Lu et al. (2020)	NN ensemble	MPPI + NN	✓
Nguyen-Tuong and Peters (2011a)	Gaussian process	Feedforward	✓
Romeres et al. (2016)	Gaussian process	feedforward + feedback	✓
Nazeer et al. (2023)	RNN	MLP	[3]
Thuruthel et al. (2019)[4]	–	FNN	–
Alessi et al. (2023)	–	FNN	–
Zhou et al. (2022)	–	Probabilistic model	✓

Table 3.1: An overview blackbox methods and their ability to be implemented incrementally. For dynamic model learning methods, the control policy specifies the control scheme in which the model is used.

add Lu2019 to chapter, add the training algorithm to the table (eg. PPO)?

[2] Used for generating training samples
[3] The incremental nature of the method is achieved by a third network, not by the dynamics model or control policy
[4] After training on generated samples

4

Learning architectures

Blackbox methods for lifelong learning are promising approaches to achieve continually adaptive control, but are hard-to-interpret. What other techniques can be implemented that are more intuitive in nature?

Well-designed control and learning architectures can either enable or facilitate lifelong learning, whether the control schemes use blackbox models or more traditional methods. For example, some techniques use two or more blackbox models, where the idea is to have each model deal with a separate challenge of the lifelong learning goal. Other architectural methods aim to expand existing control schemes or policies to enable them to continually adapt.

In this chapter, these two main strategies will be further explored in the literature.

[1] Added introduction to chapter

4.1. Traditional adaptive control architectures

4.2. Dual architectures

Under 'dual architectures' the methods are gathered that have separate modules for solving different aspects of the lifelong learning challenge. The term 'dual' is used here in a very broad sense because each module might be a grouping of various techniques with intricate architectures. However, studying the literature, it can be found that most of the methods discussed here can be broken down into two main modules. Generally, these dual architectures often exist either of a policy/control module in combination with some type of knowledge base used for storing previous experiences and informing the policy (refer to chapter 5 for details on how knowledge bases can differ in implementation), or two (or more) control policies that are each in turn capable of handling low uncertainty and high uncertainty situations.

To illustrate how these modules can be a very broad concept, consider a knowledge base: A knowledge base must first and foremost store knowledge, but additionally also maintain and update this library. Furthermore, the knowledge base should be capable of categorizing new observations in relation to its library. All these capabilities are seldom performed by a single module, but the techniques used vary wildly between different implementations. It is therefore not very interesting to differentiate between each different implementation in this chapter, but rather use the overarching term 'knowledge base' and discuss the more intricate details in chapter 5.

Having established the broad framework and significance of dual architectures, examples from literature that illustrate how these modular approaches have been effectively implemented can be discussed.

In Wang, Chen, and Dong (2020), a knowledge base stores potentially infinite sets of parametrized environments along with their respective control strategies. This repository is

used to guide the current control policy: When a new environment is encountered, the environment models in the knowledge base are evaluated on how well they fit the observations. The control policy is then initialized using the policy parameters belonging to the most similar environment found in this knowledge base, and the policy continues to learn online. In addition, environment models can be added to the library when no suitable match to the observations can be found.

Schwarz et al. (2018) utilize an architecture consisting of two modules, which they name the knowledge base and the active column. During two alternating phases, progression and compression, the active column learns a new task and the learned policy is distilled into the knowledge base, respectively. The knowledge base in this case is a policy π_t^{KB} that serves as a stable reference for the active policy π_{t+1} during progression and remains unchanged at this time. During compression, the newly learned active policy is distilled into π_t^{KB} resulting in an updated policy π_{t+1}^{KB}, and the cycle can repeat.

To combat the issue of forgetting previous policies while training on new tasks, a common approach is to store old experiences and intermittently reuse them during the training process of a new policy. One downside is that this approach can be very memory-inefficient. Shin et al. (2017) therefore propose a method where a deep generative model, a so-called *Generator*, is trained to mimic the training data provided for a task-solving model called the *Solver*. The Solver is then trained on a mix of actual training data and generated samples provided by the Generator. Likewise, when a new Generator is trained to mimic the data of the next task, some samples created by the previous Generator are mixed in. This ensures that knowledge is not forgotten, since each Generator also learns to recreate data generated by its predecessor.

An example of another approach taken when using dual architectures is found in Lu et al. (2019). Two modules, a model-free and a model-based module, work in synthesis to simulate the way human beings behave: functioning on *autopilot* most of the time and extensively planning only when we are uncertain. In the model-free module, a policy optimization algorithm (in this case, TD3 (**<empty citation>**)) uses the policy to propose an initial plan, while the standard deviation across an ensemble of value functions is used to provide a measure of uncertainty. Based on this uncertainty, a planning horizon H is set (short horizons for high uncertainty, and vice versa) and the model-based planner (MPPI) begins updating the initial plan for the given horizon, using the ensemble of value functions to evaluate its plans. During this model-based planning, any rollouts of potential plans are added to a replay buffer $\mathcal{D}_\pi$ which is subsequently used to train the TD3 policy.

citation [14] from Lu et al. (2019)

In Zhou et al. (2022) the development of a policy exists of two phases: an online interaction phase and an offline distillation phase. Each phase employs a different learning algorithm. During interaction, a policy π_0 is trained online using MPO and not necessarily paying attention to the issue of forgetting. This interaction and simultaneous training happens over multiple environments or tasks the agent encounters during its operation, and data such as the state transitions, actions, and rewards, is gathered and saved in a replay buffer $\mathcal{D}$. Once finished, the policy π_0 is discarded, and a new policy π_1 is trained offline on buffer $\mathcal{D}$ using CRR. Using this second policy, the agent is able to

4.3. Architectural expansions

'Architectural expansions' is a grouping of methods where an already functional control scheme or policy is enhanced by adding one or more modules to the scheme, enabling it to learn and/or adapt continually without actually altering the preexisting policy. The idea behind such techniques was already discussed in section 4.1, but here the focus will be on approaches that incorporate more of a learning aspect rather than simply adapting. (Chatzilygeroudis et al., 2016; Nazeer et al., 2023; Sun et al., 2011).

(Sprechmann et al., 2018)

As discussed in section 3.1. (Lu et al., 2020)

5

Storage and retrieval

(Aljundi et al., 2017; Mouret & Clune, 2015; Nguyen-Tuong & Peters, 2011a; Schulman et al., 2017; Shin et al., 2017; Sprechmann et al., 2018; Wang, Chen, & Dong, 2020; Xie & Finn, 2021).

5.1. Knowledge bases
5.2. Retrieval and Detecting distributional shifts
5.3. Maintaining the knowledge base

6

Discussion and conclusion

Todo list

redundant references? . . . 7
Add paragraph on Gijsberts and Metta (2011) . . . 7
rethink order of paragraphs? . . . 7
add Lu2019 to chapter, add the training algorithm to the table (eg. PPO)? . . . 8
citation [14] from Lu et al. (2019) . . . 10
(Sprechmann et al., 2018) . . . 10
(Lu et al., 2020) . . . 11

List of changes

Added: (Levine & Koltun, 2013) . . . 5
Added: (Haarnoja et al., 2018) . . . 6
Added: (Williams et al., 2015) . . . 6
Added: (Sayed, 2008) . . . 8
Added: (Björck, 1996) . . . 8
Added: (Landau et al., 2011) . . . 8
Added: MPC . . . 8
Commented: Added introduction to chapter . . . 9

Bibliography

Abdolmaleki, A., Springenberg, J. T., Tassa, Y., Munos, R., Heess, N., & Riedmiller, M. (2018). Maximum a Posteriori Policy Optimisation. http://arxiv.org/abs/1806.06920

Alessi, C., Hauser, H., Lucantonio, A., & Falotico, E. (2023). Learning a Controller for Soft Robotic Arms and Testing its Generalization to New Observations, Dynamics, and Tasks. *2023 IEEE International Conference on Soft Robotics (RoboSoft)*, 1–7. https://doi.org/10.1109/RoboSoft55895.2023.10121988

Aljundi, R., Chakravarty, P., & Tuytelaars, T. (2017). Expert gate: Lifelong learning with a network of experts. *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017*, *2017-Janua*, 7120–7129. https://doi.org/10.1109/CVPR.2017.753

Annaswamy, A. M., & Fradkov, A. L. (2021). A Historical Perspective of Adaptive Control and Learning. *Annual Reviews in Control*, *52*, 18–41. https://doi.org/10.1016/j.arcontrol.2021.10.014

Björck, Å. (1996, January). *Numerical Methods for Least Squares Problems*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9781611971484

Chatzilygeroudis, K., Cully, A., & Mouret, J.-B. (2016). Towards semi-episodic learning for robot damage recovery. http://arxiv.org/abs/1610.01407

Gijsberts, A., & Metta, G. (2011). Incremental learning of robot dynamics using random features. *Proceedings - IEEE International Conference on Robotics and Automation*, 951–956. https://doi.org/10.1109/ICRA.2011.5980191

Gillespie, M. T., Best, C. M., Townsend, E. C., Wingate, D., & Killpack, M. D. (2018). Learning nonlinear dynamic models of soft robots for model predictive control with neural networks. *2018 IEEE International Conference on Soft Robotics (RoboSoft)*, 39–45. https://doi.org/10.1109/ROBOSOFT.2018.8404894

Giorelli, M., Renda, F., Calisti, M., Arienti, A., Ferri, G., & Laschi, C. (2015). Learning the inverse kinetics of an octopus-like manipulator in three-dimensional space. *Bioinspiration & Biomimetics*, *10*(3), 035006. https://doi.org/10.1088/1748-3190/10/3/035006

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *35th International Conference on Machine Learning, ICML 2018*, *5*, 2976–2989. https://arxiv.org/abs/1801.01290v2

Kim, D., Kim, S.-H., Kim, T., Kang, B. B., Lee, M., Park, W., Ku, S., Kim, D., Kwon, J., Lee, H., Bae, J., Park, Y.-L., Cho, K.-J., & Jo, S. (2021). Review of machine learning methods in soft robotics (G. Gu, Ed.). *PLOS ONE*, *16*(2), e0246102. https://doi.org/10.1371/journal.pone.0246102

Landau, I. D., Lozano, R., M'Saad, M., & Karimi, A. (2011). *Adaptive Control*. Springer London. https://doi.org/10.1007/978-0-85729-664-1

Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., & Díaz-Rodríguez, N. (2020). Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, *58*, 52–68. https://doi.org/10.1016/j.inffus.2019.12.004

Levine, S., & Koltun, V. (2013). Guided policy search. *30th International Conference on Machine Learning, ICML 2013*, *28*(PART 2), 1038–1046. https://proceedings.mlr.press/v28/levine13.html

Lu, K., Grover, A., Abbeel, P., & Mordatch, I. (2020). Reset-Free Lifelong Learning with Skill-Space Planning. *ICLR 2021 - 9th International Conference on Learning Representations*. http://arxiv.org/abs/2012.03548

Lu, K., Mordatch, I., & Abbeel, P. (2019). Adaptive Online Planning for Continual Lifelong Learning. https://doi.org/10.48550/arXiv.1912.01188

Mouret, J.-B., & Clune, J. (2015). Illuminating search spaces by mapping elites. http://arxiv.org/abs/1504.04909

Nazeer, M. S., Bianchi, D., Campinoti, G., Laschi, C., & Falotico, E. (2023). Policy Adaptation using an Online Regressing Network in a Soft Robotic Arm. *2023 IEEE International Conference on Soft Robotics, RoboSoft 2023*, 1–7. https://doi.org/10.1109/RoboSoft55895.2023.10121927

Nguyen-Tuong, D., & Peters, J. (2011a). Incremental online sparsification for model learning in real-time robot control. *Neurocomputing*, *74*(11), 1859–1867. https://doi.org/10.1016/j.neucom.2010.06.033

Nguyen-Tuong, D., & Peters, J. (2011b). Model learning for robot control: a survey. *Cognitive Processing*, *12*(4), 319–340. https://doi.org/10.1007/s10339-011-0404-1

Nguyen-Tuong, D., Seeger, M., & Peters, J. (2008). Local Gaussian Process Regression for Real Time Online Model Learning and Control. *NIPS'08: Proceedings of the 21st International Conference on Neural Information Processing Systems*, 1193–1200. https://doi.org/10.5555/2981780.2981929

Nguyen-Tuong, D., Seeger, M., & Peters, J. (2009). Model Learning with Local Gaussian Process Regression. *Advanced Robotics*, *23*(15), 2015–2034. https://doi.org/10.1163/016918609X12529286896877

Romeres, D., Zorzi, M., Camoriano, R., & Chiuso, A. (2016). Online semi-parametric learning for inverse dynamics modeling. http://arxiv.org/abs/1603.05412

Sayed, A. H. (2008, March). *Adaptive Filters*. Wiley. https://doi.org/10.1002/9780470374122

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. http://arxiv.org/abs/1707.06347

Schwarz, J., Luketina, J., Czarnecki, W. M., Grabska-Barwinska, A., Teh, Y. W., Pascanu, R., & Hadsell, R. (2018). Progress & compress: A scalable framework for continual learning. *35th International Conference on Machine Learning, ICML 2018*, *10*, 7199–7208. http://arxiv.org/abs/1805.06370

Shin, H., Lee, J. K., Kim, J., & Kim, J. (2017). Continual Learning with Deep Generative Replay. http://arxiv.org/abs/1705.08690

Sprechmann, P., Jayakumar, S. M., Rae, J. W., Pritzel, A., Badia, A. P., Uria, B., Vinyals, O., Hassabis, D., Pascanu, R., & Blundell, C. (2018). Memory-based parameter adaptation. *6th International Conference on Learning Representations, ICLR 2018 - Conference Track Proceedings*. http://arxiv.org/abs/1802.10542

Sun, T., Pei, H., Pan, Y., Zhou, H., & Zhang, C. (2011). Neural network-based sliding mode adaptive control for robot manipulators. *Neurocomputing*, *74*(14-15), 2377–2384. https://doi.org/10.1016/j.neucom.2011.03.015

Thuruthel, T. G., Falotico, E., Renda, F., & Laschi, C. (2019). Model-Based Reinforcement Learning for Closed-Loop Dynamic Control of Soft Robotic Manipulators. *IEEE Transactions on Robotics*, *35*(1), 124–134. https://doi.org/10.1109/TRO.2018.2878318

Wang, Z., Chen, C., & Dong, D. (2020). Lifelong Incremental Reinforcement Learning with Online Bayesian Inference. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(8), 4003–4016. https://doi.org/10.1109/TNNLS.2021.3055499

Wang, Z., Novikov, A., Zolna, K., Springenberg, J. T., Reed, S., Shahriari, B., Siegel, N., Merel, J., Gulcehre, C., Heess, N., & de Freitas, N. (2020). Critic Regularized Regression. http://arxiv.org/abs/2006.15134

Williams, G., Aldrich, A., & Theodorou, E. (2015). Model Predictive Path Integral Control using Covariance Variable Importance Sampling. https://arxiv.org/abs/1509.01149v3%20http://arxiv.org/abs/1509.01149

Xie, A., & Finn, C. (2021). Lifelong Robotic Reinforcement Learning by Retaining Experiences. *Proceedings of Machine Learning Research*, *199*, 838–855. http://arxiv.org/abs/2109.09180

Zhou, W., Bohez, S., Humplik, J., Abdolmaleki, A., Rao, D., Wulfmeier, M., Haarnoja, T., & Heess, N. (2022). Forgetting and Imbalance in Robot Lifelong Learning with Off-policy Data. *Proceedings of Machine Learning Research*, *199*, 294–309. http://arxiv.org/abs/2204.05893